SOFTWARE DESIGN DOCUMENT

Introduction

Purpose

This Software Design Document describes how the Moderated Learning Management System (LMS) will be organized and designed before implementation

Scope

The system will be developed as a C++17 application using SQLite for persistent data storage. It will operate on a single school computer

System Architecture

The proposed architecture is a **layered architecture**. The system is divided into layers, with each layer responsible for a particular part of the application.

The four layers are:

Presentation layer: Displays menus, receives input and presents results to users.

Application and business-logic layer: Checks permissions and applies rules for assignments, submissions, extensions, marks and moderation.

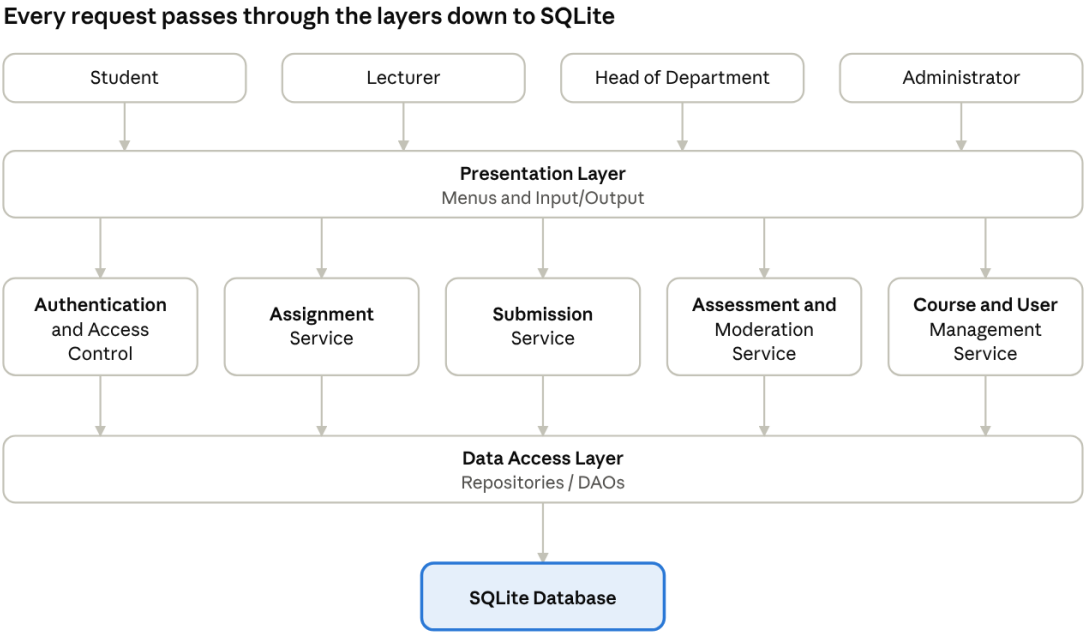
Data-access layer: Handles database operations and separates SQL operations from the business logic.

Database layer: Stores user accounts, courses, assignments, submissions, marks, feedback and audit records in SQLite.

The application will use a console interface or a simple desktop interface, depending on the final implementation choice. Neither requires a web or mobile interface.

Component diagram

The diagram below shows the main components and their relationships.



component diagram · 4 actors, 4 layers

Component diagram of the Moderated LMS.

Component responsibilities

Component	Responsibility
Presentation layer	Displays menus, accepts user input and shows messages, assignments, marks and feedback.
Authentication and access control	Verifies login credentials and checks whether a user's role permits an action.
Assignment service	Creates assignments, manages deadlines and handles individual extensions.
Submission service	Accepts submissions and resubmissions, records submission times and determines lateness.
Assessment and moderation service	Records marks and feedback, applies late-penalty rules, supports moderation and controls mark release.
Course and user management service	Manages accounts, departments, courses, lecturer assignments and student enrolments.
Data-access layer	Reads and writes information through repositories or data-access objects using parameterised SQL queries.
SQLite database	Stores application data persistently on the school's computer or local network.

Design rationale

The layered architecture was selected because the system has several responsibilities that should not be mixed together. For example, displaying a menu should not also involve calculating a late penalty or writing SQL statements directly.

Separating the layers offers the following benefits:

Maintainability: Changes to the interface can be made without rewriting the database logic.

Testability: Business rules can be tested separately from the user interface.

Security: Access checks can be performed before protected operations are allowed.

Data integrity: Database operations are handled through a dedicated data-access layer.

Offline operation: The application can use a local SQLite database without depending on internet access.

Alternative considered and rejected

A single, unstructured application in which menus, business rules and SQL queries are mixed together was considered but rejected.

Although this approach might initially require fewer files, it would make the application harder to test, maintain and modify. A change to one function could unintentionally affect unrelated functions. It would also make it harder to ensure that database access follows consistent security and validation rules.

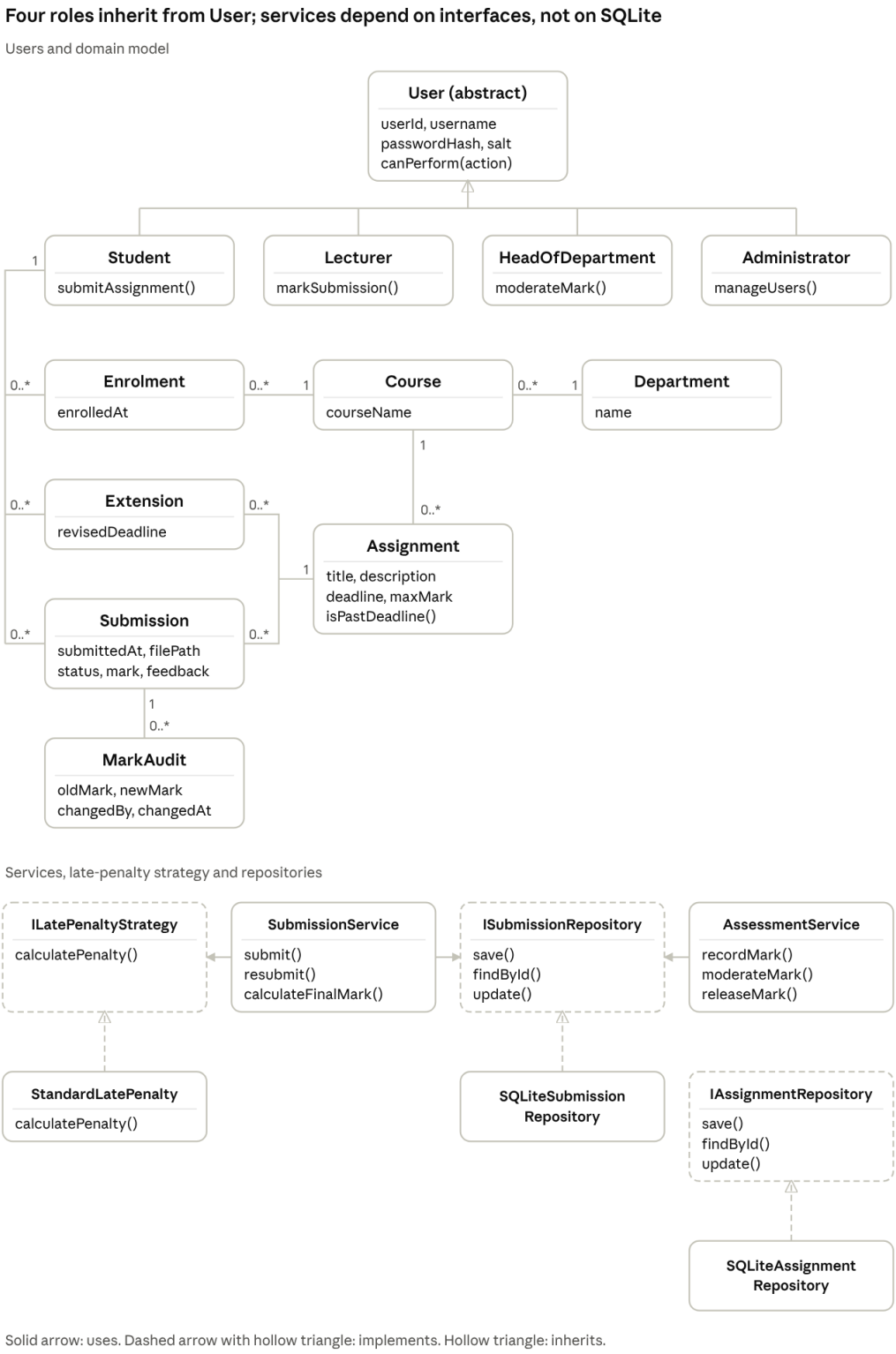
Detailed Class Design

The class design describes the main objects in the system, the information they store, the operations they perform and the relationships between them.

The design uses inheritance for the different user roles, interfaces for replaceable behaviour and separate classes for business rules and database access.

Class diagram

The following diagram represents the proposed design-level class diagram.



class diagram · users, domain model, services and repositories

Detailed class diagram of the Moderated LMS.

Main classes and responsibilities

Class	Responsibility
`User`	Abstract base class containing shared user information and common access-control behaviour.
`Student`	Represents a student who views assignments, submits work and views released results.
`Lecturer`	Represents a lecturer who creates assignments, grants individual extensions and marks submissions.
`HeadOfDepartment`	Reviews and moderates marks and authorises their release.
`Administrator`	Manages users, courses, departments and enrolments.
`Assignment`	Stores assignment details, maximum mark and deadline.
`Submission`	Stores a student's submitted work, submission time, status, mark and feedback.
`Extension`	Records an individual student's revised deadline and the lecturer who granted it.
`Course`	Represents a course and connects it to its department and assignments.
`Department`	Represents a school department.
`Enrolment`	Connects students to the courses in which they are enrolled.
`MarkAudit`	Records mark changes, including who changed the mark, when, the previous mark, the new mark and the reason.
`SubmissionService`	Coordinates submissions, resubmissions and late-penalty calculations.
`AssessmentService`	Coordinates marking, moderation and mark release.
Repository interfaces and implementations	Separate data-access operations from business logic.

Design patterns

Strategy pattern — late-penalty calculation

The Strategy pattern allows the late-penalty calculation to be represented by a separate class implementing `ILatePenaltyStrategy`.

The `StandardLatePenalty` class applies the agreed rule: deduct 10% of the available mark for each day late, up to three days; work more than three days late receives zero. An applicable individual extension changes the deadline used to determine lateness.

For example, for an assignment worth 100 marks, a submission one day late receives a 10-mark penalty before the penalty is applied to the mark earned.

This pattern was selected because it separates the penalty rule from submission handling. If the school's policy changes later, a different strategy can be introduced without rewriting the entire submission service.

Repository/DAO pattern — database access

The Repository or Data Access Object (DAO) pattern separates database operations from business logic. For example, `SubmissionService` uses the `ISubmissionRepository` interface instead of executing SQL statements itself. `SQLiteSubmissionRepository` implements the interface using SQLite.

This pattern was selected because it:

Keeps SQL queries out of business classes.

Makes database operations easier to maintain.

Supports testing business logic independently of SQLite.

Encourages parameterised queries and consistent data access.

Application of SOLID principles

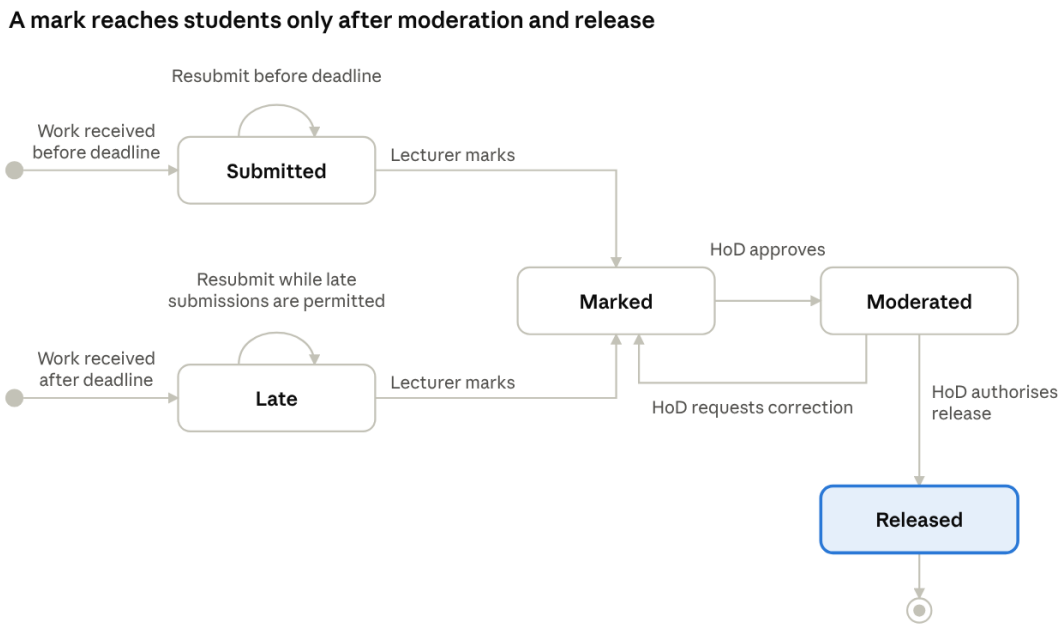
Principle	Application in the LMS
Single Responsibility Principle (SRP)	`SubmissionService` handles submission workflows, while repository classes handle database access.
Open/Closed Principle (OCP)	A new late-penalty strategy can be added through `ILatePenaltyStrategy` without changing the submission service's overall design.
Liskov Substitution Principle (LSP)	Each user-role subclass should be usable wherever a `User` is expected without breaking the expected behaviour.
Interface Segregation Principle (ISP)	Small interfaces, such as `ISubmissionRepository`, expose only the operations needed for that responsibility.
Dependency Inversion Principle (DIP)	Business services depend on repository and strategy interfaces rather than directly depending on concrete SQLite classes.

Behavioural Models

Behavioural models show how the LMS behaves over time. They describe the lifecycle of a submission, the complete workflow from submission to released marks and the interactions between objects.

Submission state machine diagram

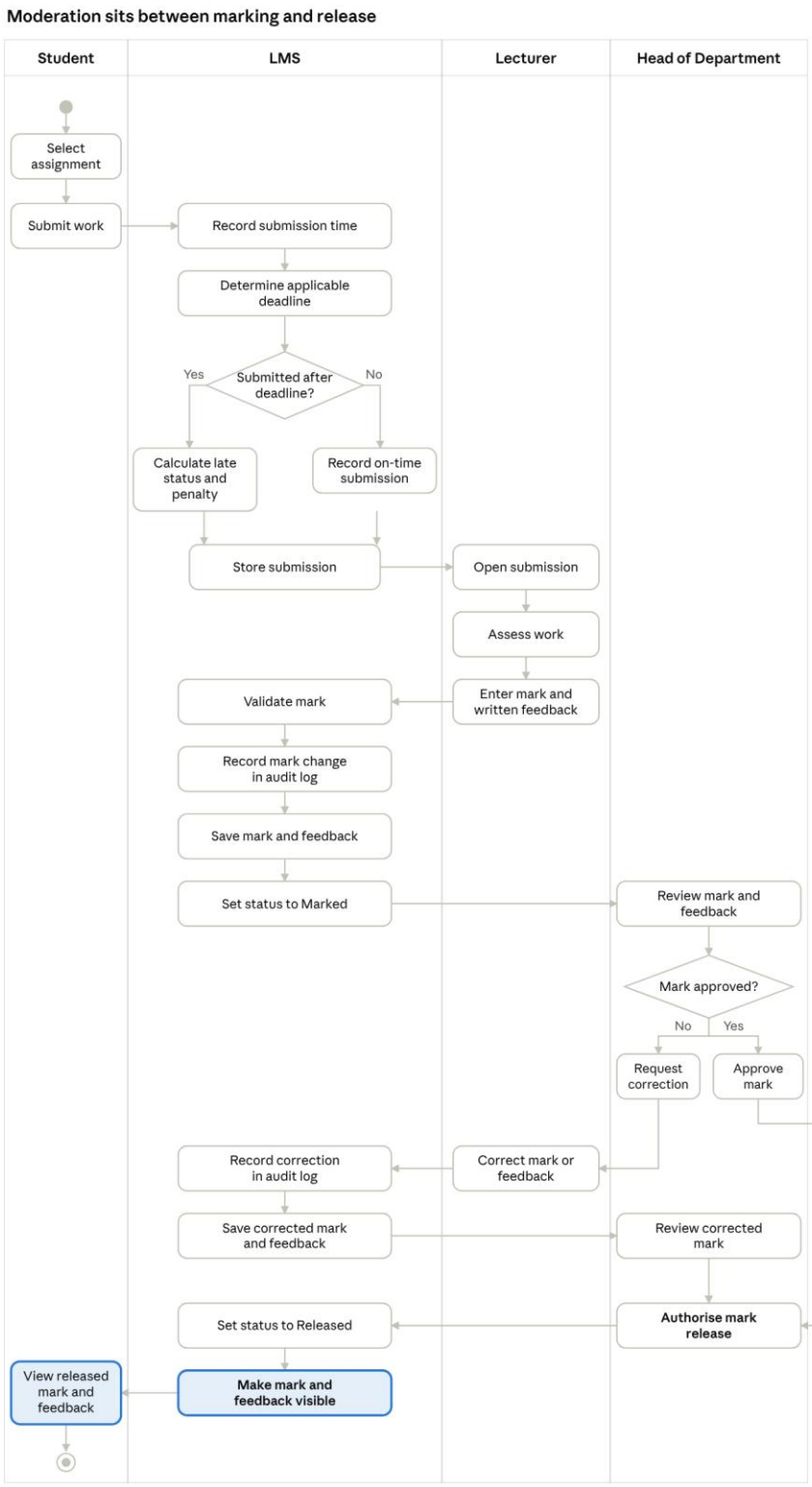
The state machine shows the states a submission can pass through from the moment it is submitted until its mark is released.



submission lifecycle · 5 states

Submission lifecycle state machine.
Explanation
A student submits work. The system determines whether it was submitted before or after the applicable deadline.
Work is recorded as `Submitted` or `Late`.
A student may resubmit before the deadline, replacing the previous submission according to the resubmission rule.
The lecturer marks the work and provides written feedback.
The submission moves to `Marked`.
The HoD reviews the mark. If a correction is needed, it returns to `Marked` for further action.
When the HoD approves the mark, it moves to `Moderated`.
The HoD authorises release, moving the submission to `Released`.
The student can view the released mark and feedback.
A mark must not be visible to the student before the HoD authorises its release. Every mark change must be recorded in the audit log.

Activity diagram with swimlanes
The activity diagram shows the end-to-end workflow from submission to release. The swimlanes identify which actor or system component performs each action.



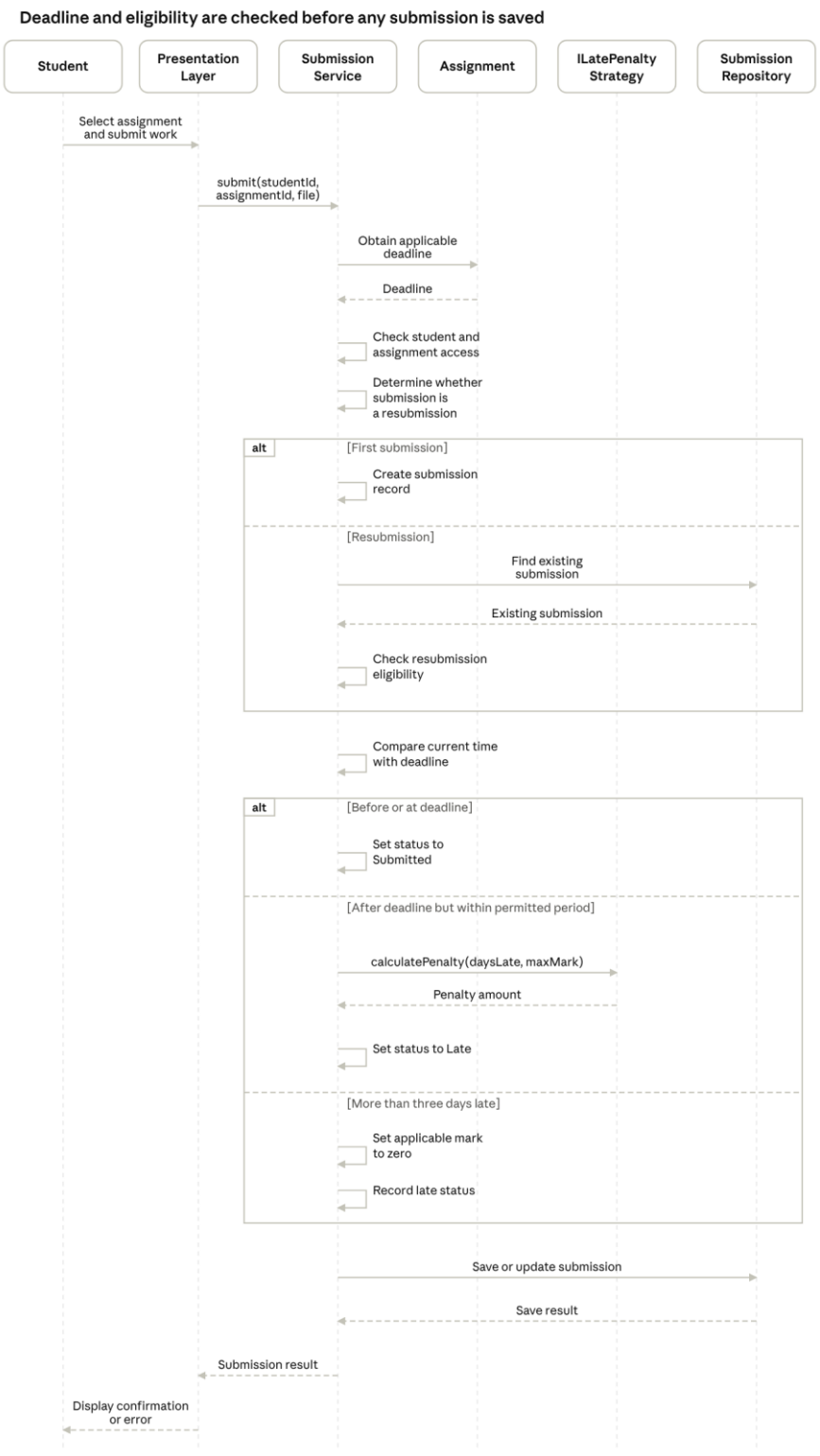
swimlane activity diagram · 4 lanes, 2 decisions

Activity diagram for submission, marking, moderation and release.

Explanation

The student begins the process by submitting work. The LMS records the submission time and determines whether the work is late. The lecturer then marks the submission and gives written feedback.
Before the mark can be released, the HoD must review it. If a correction is requested, the lecturer corrects the mark or feedback and the change is recorded. Once the HoD approves the result, the LMS releases the mark and feedback to the student.
This workflow ensures that marks cannot bypass the moderation process.

Sequence diagram — Submit assignment
This sequence diagram shows the interactions required to submit work. It includes the late-submission and resubmission paths.



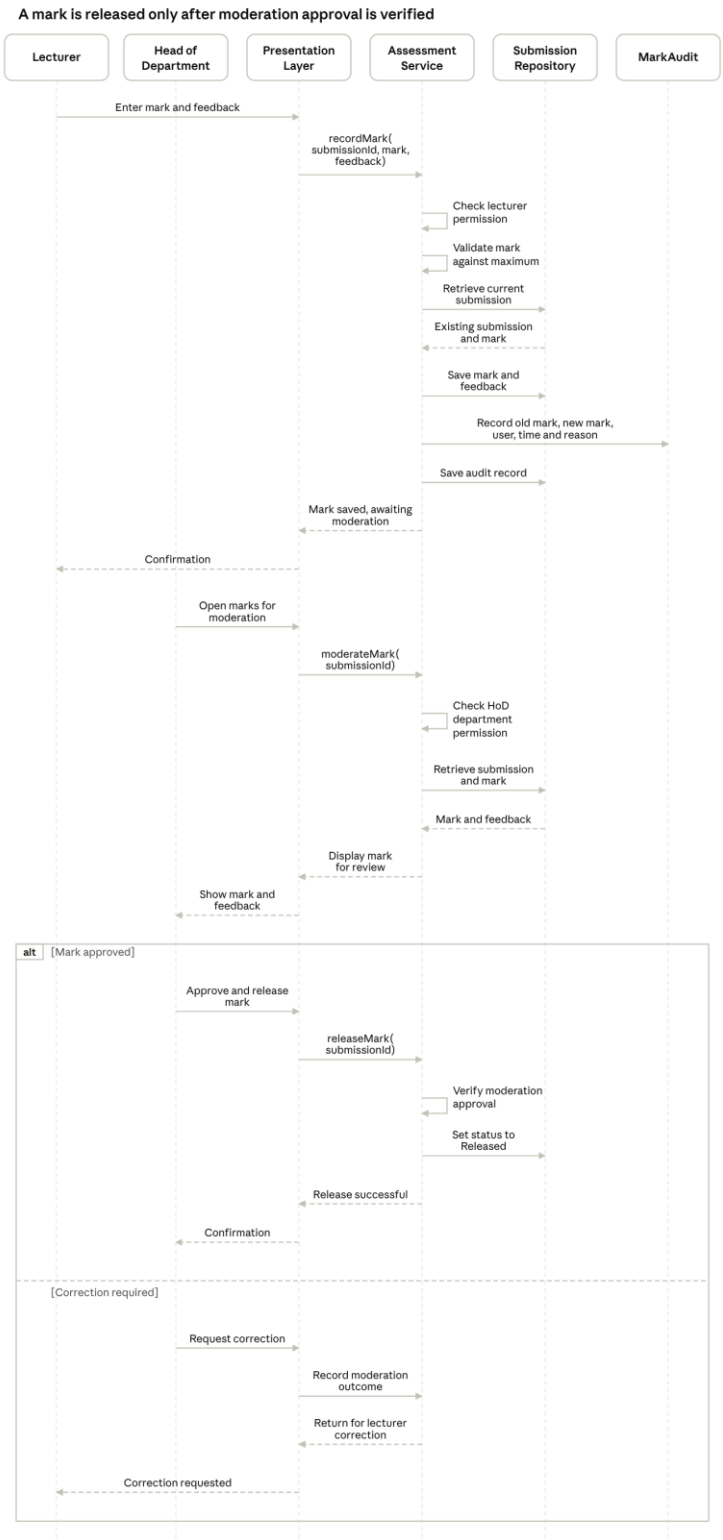
sequence diagram · submit and resubmit, 2 alt frames

Sequence diagram for submitting and resubmitting work.

Explanation

The student selects an assignment and submits work through the presentation layer. The submission service checks the applicable deadline, the student's eligibility and whether the action is a first submission or a resubmission.
For a resubmission, the service retrieves the existing record and checks whether resubmission is permitted. The service then determines lateness and applies the relevant rule. Finally, it saves the submission and returns a confirmation or error message.
An applicable individual extension must be used when determining the deadline. The system should not accept a resubmission simply because the student has selected an existing submission; it must check the eligibility rules.

Sequence diagram — Moderate and release marks
This sequence diagram shows how marks are recorded, reviewed and released.

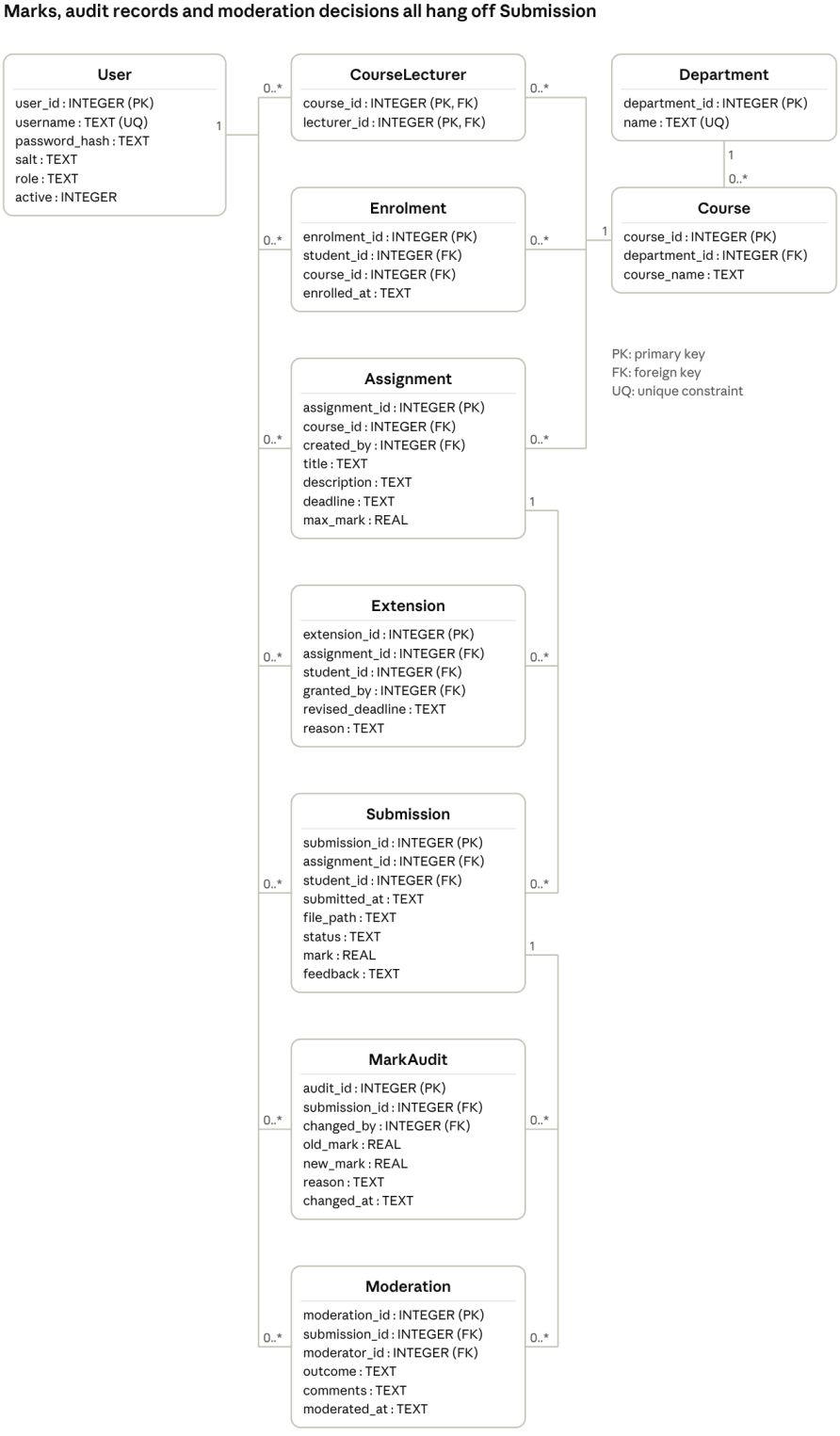


sequence diagram · record, moderate and release marks
Sequence diagram for moderating and releasing marks.
Explanation

The lecturer enters a mark and written feedback. The assessment service checks permissions, validates the mark and records the change in the audit log.
The HoD then reviews the mark and feedback for courses in their department. If the mark is approved, the release operation checks that moderation approval exists before changing the status to `Released`. If a correction is needed, the mark is returned to the lecturer for correction.
The system must prevent students from accessing unreleased marks.

Database Design
Database approach
The LMS will use SQLite as its relational database. The database will store information needed to manage users, departments, courses, enrolments, assignments, submissions, individual extensions and mark changes.
The database will be accessed through the data-access layer rather than directly by presentation or business-logic classes. SQL statements that contain user-supplied values will use parameterised queries.

Entity-Relationship Diagram (ERD)



entity-relationship diagram · 10 tables

Entity-Relationship Diagram of the LMS database.

Data dictionary

The data dictionary defines the main database tables and their fields.

Table	Important fields	Purpose
`User`	`user_id`, `username`, `password_hash`, `salt`, `role`, `active`	Stores account information and user roles.
`Department`	`department_id`, `name`	Stores school departments.
`Course`	`course_id`, `department_id`, `course_name`	Stores courses and their departments.
`CourseLecturer`	`course_id`, `lecturer_id`	Connects lecturers to the courses they teach.
`Enrolment`	`enrolment_id`, `student_id`, `course_id`, `enrolled_at`	Records student enrolment in courses.
`Assignment`	`assignment_id`, `course_id`, `created_by`, `title`, `description`, `deadline`, `max_mark`	Stores assignment details and deadlines.
`Extension`	`extension_id`, `assignment_id`, `student_id`, `granted_by`, `revised_deadline`, `reason`	Records individual deadline extensions.
`Submission`	`submission_id`, `assignment_id`, `student_id`, `submitted_at`, `file_path`, `status`, `mark`, `feedback`	Stores submitted work and its assessment details.
`MarkAudit`	`audit_id`, `submission_id`, `changed_by`, `old_mark`, `new_mark`, `reason`, `changed_at`	Keeps a history of mark changes.
`Moderation`	`moderation_id`, `submission_id`, `moderator_id`, `outcome`, `comments`, `moderated_at`	Records HoD moderation decisions.

Data types: `INTEGER` stores whole numbers and identifiers, `TEXT` stores text and date/time values, and `REAL` stores numerical marks.
Keys: A primary key uniquely identifies a row. A foreign key links a row to a related row in another table. A unique constraint prevents duplicate values where uniqueness is required.

SQL schema
The following SQL defines the proposed database tables, primary keys, foreign keys and important constraints.

```
PRAGMA foreign_keys = ON;

CREATE TABLE User (
  user_id INTEGER PRIMARY KEY,
  username TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  salt TEXT NOT NULL,
  role TEXT NOT NULL CHECK (
    role IN ('Student', 'Lecturer', 'HoD', 'Administrator')
  ),
  active INTEGER NOT NULL DEFAULT 1 CHECK (active IN (0, 1))
);

CREATE TABLE Department (
  department_id INTEGER PRIMARY KEY,
  name TEXT NOT NULL UNIQUE
);
```

```
CREATE TABLE Course (
   course_id INTEGER PRIMARY KEY,
   department_id INTEGER NOT NULL,
   course_name TEXT NOT NULL,
   FOREIGN KEY (department_id)
      REFERENCES Department(department_id)
);

CREATE TABLE CourseLecturer (
   course_id INTEGER NOT NULL,
   lecturer_id INTEGER NOT NULL,
   PRIMARY KEY (course_id, lecturer_id),
   FOREIGN KEY (course_id) REFERENCES Course(course_id),
   FOREIGN KEY (lecturer_id) REFERENCES User(user_id)
);

CREATE TABLE Enrolment (
   enrolment_id INTEGER PRIMARY KEY,
   student_id INTEGER NOT NULL,
   course_id INTEGER NOT NULL,
   enrolled_at TEXT NOT NULL,
   UNIQUE (student_id, course_id),
   FOREIGN KEY (student_id) REFERENCES User(user_id),
   FOREIGN KEY (course_id) REFERENCES Course(course_id)
);

CREATE TABLE Assignment (
   assignment_id INTEGER PRIMARY KEY,
   course_id INTEGER NOT NULL,
   created_by INTEGER NOT NULL,
   title TEXT NOT NULL,
   description TEXT NOT NULL,
   deadline TEXT NOT NULL,
   max_mark REAL NOT NULL CHECK (max_mark > 0),
   FOREIGN KEY (course_id) REFERENCES Course(course_id),
   FOREIGN KEY (created_by) REFERENCES User(user_id)
);

CREATE TABLE Extension (
   extension_id INTEGER PRIMARY KEY,
   assignment_id INTEGER NOT NULL,
   student_id INTEGER NOT NULL,
   granted_by INTEGER NOT NULL,
   revised_deadline TEXT NOT NULL,
   reason TEXT NOT NULL,
   UNIQUE (assignment_id, student_id),
   FOREIGN KEY (assignment_id) REFERENCES Assignment(assignment_id),
   FOREIGN KEY (student_id) REFERENCES User(user_id),
```

```
    FOREIGN KEY (granted_by) REFERENCES User(user_id)
);

CREATE TABLE Submission (
    submission_id INTEGER PRIMARY KEY,
    assignment_id INTEGER NOT NULL,
    student_id INTEGER NOT NULL,
    submitted_at TEXT NOT NULL,
    file_path TEXT NOT NULL,
    status TEXT NOT NULL CHECK (
        status IN ('Submitted', 'Late', 'Marked',
                'Moderated', 'Released')
    ),
    mark REAL,
    feedback TEXT,
    UNIQUE (assignment_id, student_id),
    CHECK (mark IS NULL OR mark >= 0),
    FOREIGN KEY (assignment_id) REFERENCES Assignment(assignment_id),
    FOREIGN KEY (student_id) REFERENCES User(user_id)
);

CREATE TABLE MarkAudit (
    audit_id INTEGER PRIMARY KEY,
    submission_id INTEGER NOT NULL,
    changed_by INTEGER NOT NULL,
    old_mark REAL,
    new_mark REAL NOT NULL CHECK (new_mark >= 0),
    reason TEXT NOT NULL,
    changed_at TEXT NOT NULL,
    FOREIGN KEY (submission_id) REFERENCES Submission(submission_id),
    FOREIGN KEY (changed_by) REFERENCES User(user_id)
);

CREATE TABLE Moderation (
    moderation_id INTEGER PRIMARY KEY,
    submission_id INTEGER NOT NULL,
    moderator_id INTEGER NOT NULL,
    outcome TEXT NOT NULL CHECK (
        outcome IN ('Approved', 'CorrectionRequired')
    ),
    comments TEXT,
    moderated_at TEXT NOT NULL,
    FOREIGN KEY (submission_id) REFERENCES Submission(submission_id),
    FOREIGN KEY (moderator_id) REFERENCES User(user_id)
);
```

Schema rules and implementation notes

Foreign-key enforcement must be enabled for database connections.

User roles must be checked by the application before protected operations are performed. The role field alone does not grant permission.
The application must verify that students, lecturers and HoDs are associated with the relevant courses or departments before they perform course-specific actions.
A mark must not exceed the assignment's maximum mark. This check requires application validation or an appropriate database trigger.
Mark changes must be recorded in `MarkAudit` as part of the same database transaction as the corresponding update.
The schema permits a submission to have no mark until it has been assessed. The application must enforce the moderation and release workflow.
The `UNIQUE (assignment_id, student_id)` constraint represents one current submission record per student per assignment. Resubmission updates that record. If a complete history of every submitted file is required, the schema must be extended to store separate submission versions.
The `Extension` table records individual extensions, as required in the base system. Class-wide extensions are addressed later by the assignment's maintenance change request.

Database normalisation
The database is designed to satisfy **Third Normal Form (3NF)**.
First Normal Form (1NF)
Each table stores individual values in its fields. For example, each user has one username and one role, rather than storing several users in one field.
Second Normal Form (2NF)
The tables avoid attributes that depend on only part of a composite key. In `CourseLecturer`, the composite key consists of `course_id` and `lecturer_id`; the table contains only the relationship between those two identifiers.
Third Normal Form (3NF)
Non-key attributes depend on the key, the whole key and nothing but the key. For example, the department name is stored in `Department`, not repeatedly in every course or assignment record. A course refers to its department through `department_id`.
This separation reduces duplication and helps prevent inconsistent updates.

Mapping database tables to classes

Database table	Corresponding class or classes	Explanation
`User`	`User`, `Student`, `Lecturer`, `HeadOfDepartment`, `Administrator`	One database table stores common account details and the role; the application uses inheritance to represent role-specific behaviour.
`Department`	`Department`	Stores department information.
`Course`	`Course`	Stores course information and its department relationship.
`CourseLecturer`	Course–lecturer association	Represents which lecturers teach which courses.
`Enrolment`	`Enrolment`	Represents the many-to-many relationship between students and courses.
`Assignment`	`Assignment`	Stores assignment details.
`Extension`	`Extension`	Stores individual deadline extensions.

Database table	Corresponding class or classes	Explanation
`Submission`	`Submission`	Stores the current submission and assessment details.
`MarkAudit`	`MarkAudit`	Stores the history of mark changes.
`Moderation`	Moderation record used by `AssessmentService`	Stores moderation decisions and comments.